

Deep Learning for NLP

Student name: *sdi2200160*

Course: *Artificial Intelligence II (YS19 M226 M262 M325 M405)*

Semester: *Spring Semester 2025-2026*

Contents

1	Abstract	3
2	Task and Data	3
2.1	Dataset and Labels	3
2.2	From Homework 1 to Homework 2	4
2.3	Preprocessing and Input Construction	4
2.4	Compact Exploratory Analysis	4
3	Method	5
3.1	Shared Pipeline Structure	5
3.2	Transformer Components	5
3.3	Training Loop	5
4	Experimental Protocol	6
4.1	Compared Models	6
4.2	Kaggle Notebooks	6
4.3	Validation Strategy	6
4.4	Default Training Recipe	7
4.5	Tiny Uniform Sanity Sweep	7
5	Evaluation Protocol	9
6	Results	9
6.1	Homework 1 Reference Point	9
6.2	Transformer Test Results	9
6.3	Partial Interpretation	10
7	Subgroup Analysis	11
8	Error Analysis	13
8.1	Observed Error Patterns	13
8.2	Representative Mistakes	14
9	Discussion	15
9.1	Small Non-Improving Changes	15

10 Conclusion

16

1. Abstract

We study **response clarity classification** on the QEvasion political interview dataset [1]. Each example is a question–answer pair, and the target label is one of *Clear Reply*, *Ambivalent*, or *Clear Non-Reply*. Homework 1 approached this task with non-contextual representations such as TF-IDF and averaged GloVe embeddings combined with Logistic Regression. Homework 2 keeps the same high-level pipeline abstraction, but replaces the source encoder with a pretrained transformer tokenizer and replaces the linear model with a fine-tuned transformer sequence classifier.

The goal of this report is not to present an aggressively optimized system. Instead, we document a **simple and principled transformer baseline**: raw question–answer pairs, sentence-pair tokenization, one fixed stratified validation split, AdamW optimization, class-weighted loss, gradient clipping, and checkpoint selection by validation macro F1. The BERT-base, DistilBERT, and DeBERTa-v3 experiments have completed successfully in the sense that all notebooks produced outputs. However, the DeBERTa-v3 fine-tuning run was numerically unstable: its losses became nan, and the final model collapsed to predicting only *Ambivalent*. We therefore report it transparently as a failed fine-tuning run rather than as a competitive model.

2. Task and Data

2.1. Dataset and Labels

We use the QEvasion dataset from Hugging Face [1]. Each supervised example contains:

- a segmented journalist question,
- the corresponding interview_answer,
- the target label clarity_label.

The task is three-way classification:

- **Clear Reply**,
- **Ambivalent**,
- **Clear Non-Reply**.

As in Homework 1, we use the official train/test split provided by the dataset. The training split is used for fitting and internal validation, while the official test split is used only for final evaluation and Kaggle submission export.

Split	Instances	Usage
Train	3448	fitting + internal validation
Test	308	final reporting only

Table 1: Dataset split summary used in the completed transformer runs.

2.2. From Homework 1 to Homework 2

The conceptual pipeline remains the same across the two assignments:

1. preprocess the raw input,
2. encode the source text,
3. encode the target labels,
4. fit a classifier,
5. decode predictions and evaluate.

This continuity is deliberate. Homework 2 should read as a clean extension of Homework 1 rather than a completely different implementation. The important change is therefore the *representation*: Homework 1 used concatenated text plus non-contextual encoders, while Homework 2 keeps the question and answer as a pair and lets the transformer model learn contextual interactions between them.

2.3. Preprocessing and Input Construction

Preprocessing is intentionally lighter than in Homework 1. We do *not* apply lemmatization, stopword removal, or handcrafted punctuation normalization. Instead, we keep the raw question and answer text, replace missing values with empty strings, and strip only outer whitespace.

This design is natural for transformer fine-tuning because the pretrained checkpoints were trained on raw text and expect model-specific special tokens and subword segmentation. Each example is passed to the tokenizer as a sentence pair:

(question, answer)

which is internally converted to the checkpoint-specific format. For BERT-like models, this is conceptually similar to:

[CLS] question [SEP] answer [SEP].

Relative to Homework 1, this is a principled simplification: instead of manually engineering lexical features, we rely on the pretrained tokenizer and contextual encoder to model the interaction between the question and the response.

2.4. Compact Exploratory Analysis

The notebook keeps the exploratory analysis compact and focused on factors that matter for model behavior:

- class imbalance,
- question length distribution,
- answer length distribution,

- combined text-length variation across splits.

As in Homework 1, class imbalance remains important. In the completed transformer runs, the training labels required class weights 0.5634 for *Ambivalent*, 3.2285 for *Clear Non-Reply*, and 1.0925 for *Clear Reply*. This indicates that *Clear Non-Reply* is the smallest training class and receives the strongest upweighting. The test split is similarly imbalanced, with 206 *Ambivalent*, 23 *Clear Non-Reply*, and 79 *Clear Reply* examples. This motivates both macro-averaged evaluation and class-weighted training.

3. Method

3.1. Shared Pipeline Structure

We reuse the same thin Classifier wrapper idea from Homework 1. The wrapper coordinates:

- preprocessing,
- source encoding,
- target encoding,
- model fitting,
- prediction and scoring.

This matters pedagogically. The new report can then argue that the conceptual pipeline is stable while only the encoder and model components change.

3.2. Transformer Components

Homework 2 introduces two new central components:

- `TokenizerEncoder`, which wraps a Hugging Face `AutoTokenizer` and converts each (question, answer) pair into transformer inputs;
- `TransformerModel`, which wraps the Hugging Face sequence-classification auto-model and fine-tunes it using an explicit PyTorch loop.

We do *not* use the Hugging Face Trainer API. The training loop is written manually, which keeps the mechanics visible and stays aligned with the assignment's spirit.

3.3. Training Loop

The training loop is intentionally minimal:

- optimizer: AdamW [4],
- loss: weighted cross-entropy,
- model selection: best validation macro F1,

- regularization: weight decay and gradient clipping,
- device: CUDA only.

Class weights are computed from the training labels using the standard balanced-class formula:

$$w_c = \frac{N}{K \cdot n_c},$$

where N is the number of training examples, K is the number of classes, and n_c is the number of training examples belonging to class c .

After each epoch, we evaluate on the fixed validation split and keep the best checkpoint by validation macro F1. This means the reported final model is not merely the last epoch, but the best epoch observed during training.

4. Experimental Protocol

4.1. Compared Models

We compare the three required pretrained checkpoints:

- bert-base-uncased [2],
- distilbert-base-uncased [5],
- microsoft/deberta-v3-base [3].

All three are fine-tuned as three-class sequence classifiers under the same overall recipe. At the time of this report, all required notebooks have produced Kaggle outputs. BERT and DistilBERT trained normally. DeBERTa-v3 produced nan losses throughout the sweep and final run, so its numbers should be interpreted as an unstable run rather than evidence that the architecture itself is weak.

4.2. Kaggle Notebooks

The executed Kaggle notebooks used for the reported runs are:

- BERT: [Kaggle notebook for bert-base-uncased](#).
- DistilBERT: [Kaggle notebook for distilbert-base-uncased](#).
- DeBERTa-v3: [Kaggle notebook for microsoft/deberta-v3-base](#).

4.3. Validation Strategy

Homework 1 used 3-fold cross-validation because the classical models were relatively cheap to train. Homework 2 uses transformer fine-tuning, so repeating full training several times per hyperparameter setting would make the workflow much heavier. We therefore replace cross-validation with a single **15% stratified validation split** taken from the official training split.

This yields a simple and defensible procedure:

- 85% of the official training split is used for gradient updates,
- 15% is used for validation,
- the official test split is never used for model selection.

4.4. Default Training Recipe

The shared default recipe is:

- learning rate: 2×10^{-5} ,
- batch size: 16,
- weight decay: 0.01,
- maximum gradient norm: 1.0,
- validation fraction: 0.15,
- random seed: 42,
- class weighting: enabled.

Parameter	Value	Role
Learning rate	2×10^{-5}	conservative fine-tuning step size
Batch size	16	fits standard Kaggle GPU memory budgets
Weight decay	0.01	mild regularization
Max gradient norm	1.0	stabilizes training
Validation split	15% stratified	checkpoint selection
Class weights	enabled	partially offsets imbalance
Random seed	42	reproducibility

Table 2: Shared default training recipe.

4.5. Tiny Uniform Sanity Sweep

The only tuning we allow is the same small sweep for each required model:

- epochs $\in \{3, 4\}$,
- maximum sequence length $\in \{128, 256, 384\}$.

This produces 6 runs per model, or 18 runs in total across BERT, DistilBERT, and DeBERTa. The sweep is intentionally small. Its role is not exhaustive optimization, but a sanity check over two fundamental choices:

- **training duration**, via the number of epochs;
- **visible context**, via the maximum input length.

Because the same 2×3 search space is used for all three models, the tuning budget remains uniform across architectures.

Model	Best Epochs	Best Max Length	Best Validation Macro F1	Notes
BERT	4	256	0.6657	Best completed run
DistilBERT	4	256	0.6229	Same selected recipe as BERT
DeBERTa-v3	4	384	0.2476	Unstable run; losses were nan

Table 3: Best configuration selected by the tiny sweep. DeBERTa-v3 is included for completeness, but its run was numerically unstable.

BERT Sweep Setting	Best Epoch	Validation Macro F1	Rank
4 epochs, max length 256	4	0.6657	1
4 epochs, max length 384	4	0.6655	2
4 epochs, max length 128	2	0.6593	3
3 epochs, max length 128	2	0.6593	4
3 epochs, max length 384	3	0.6586	5
3 epochs, max length 256	3	0.6558	6

Table 4: Full tiny sweep for the completed BERT experiment.

DistilBERT Sweep Setting	Best Epoch	Validation Macro F1	Rank
4 epochs, max length 256	4	0.6229	1
4 epochs, max length 384	4	0.5918	2
3 epochs, max length 256	3	0.5742	3
3 epochs, max length 384	3	0.5643	4
4 epochs, max length 128	2	0.5523	5
3 epochs, max length 128	2	0.5523	6

Table 5: Full tiny sweep for the completed DistilBERT experiment.

DeBERTa-v3 Sweep Setting	Best Epoch	Validation Macro F1	Rank
4 epochs, max length 384	1	0.2476	1
3 epochs, max length 384	1	0.2476	2
4 epochs, max length 256	1	0.2476	3
3 epochs, max length 256	1	0.2476	4
4 epochs, max length 128	1	0.2476	5
3 epochs, max length 128	1	0.2476	6

Table 6: Full tiny sweep for DeBERTa-v3. Every run produced nan losses and the same validation macro F1, so the selected configuration is not meaningful.

5. Evaluation Protocol

We report the same metrics as in Homework 1:

- Accuracy,
- Macro Precision,
- Macro Recall,
- Macro F1.

Macro F1 remains the primary metric because it treats all classes equally and is therefore more informative than plain accuracy on an imbalanced three-class task.

In addition to overall scores, the notebook also produces:

- confusion matrices,
- learning curves for train loss, validation loss, and validation macro F1,
- subgroup results by question length and answer length,
- representative misclassifications for qualitative error analysis.

6. Results

6.1. Homework 1 Reference Point

We do not rerun the classical baselines inside Homework 2. Instead, we compare against the main Homework 1 reference values already reported there:

Homework 1 Model	Macro F1
TF-IDF + Logistic Regression	0.45
GloVe Wiki + Logistic Regression	0.39
GloVe Twitter + Logistic Regression	0.38

Table 7: Homework 1 reference results used for cross-homework comparison.

These values give the baseline that Homework 2 is trying to improve upon. The strongest classical point of comparison is TF-IDF + Logistic Regression with macro F1 0.45.

Model	Accuracy	Macro Precision	Macro Recall	Macro F1
BERT	0.6299	0.5372	0.5879	0.5561
DistilBERT	0.5779	0.4881	0.5620	0.5066
DeBERTa-v3	0.6688	0.2229	0.3333	0.2672

Table 8: Final transformer results on the official test split. DeBERTa-v3 has high accuracy only because it predicts the majority class for every test example.

BERT Class	Precision	Recall	F1	Support
Ambivalent	0.7692	0.6796	0.7216	206
Clear Non-Reply	0.4062	0.5652	0.4727	23
Clear Reply	0.4362	0.5190	0.4740	79

Table 9: Per-class BERT results on the official test split.

DistilBERT Class	Precision	Recall	F1	Support
Ambivalent	0.7561	0.6019	0.6703	206
Clear Non-Reply	0.3023	0.5652	0.3939	23
Clear Reply	0.4059	0.5190	0.4556	79

Table 10: Per-class DistilBERT results on the official test split.

6.2. Transformer Test Results

6.3. Partial Interpretation

The completed BERT run achieved macro F1 0.5561, which is higher than the Homework 1 TF-IDF baseline of 0.45 by +0.1061 and higher than the best Homework 1 GloVe baseline by +0.1661. DistilBERT also improved over the Homework 1 baselines, reaching macro F1 0.5066. BERT is the strongest completed transformer, ahead of DistilBERT by 0.0495 macro F1. DeBERTa-v3 reached only macro F1 0.2672, below all Homework 1 baselines, because the run became numerically unstable and predicted only the majority class. Thus the useful comparison is primarily between BERT, DistilBERT, and the Homework 1 baselines.

Per-class scores show that the aggregate gain does not solve the task completely. BERT performs best on *Ambivalent* with F1 0.7216, while *Clear Non-Reply* remains its hardest class with F1 0.4727. DistilBERT has lower *Ambivalent* and *Clear Reply* F1, and also lower *Clear Non-Reply* F1 at 0.3939. DeBERTa-v3 obtains F1 0.0000 on both non-majority classes because it never predicts them. The low scores for *Clear Non-Reply* are consistent with its small test support of only 23 examples and with the qualitative difficulty of distinguishing a genuinely non-responsive answer from a partially responsive one.

DeBERTa-v3 Class	Precision	Recall	F1	Support
Ambivalent	0.6688	1.0000	0.8016	206
Clear Non-Reply	0.0000	0.0000	0.0000	23
Clear Reply	0.0000	0.0000	0.0000	79

Table 11: Per-class DeBERTa-v3 results on the official test split. The model predicted only *Ambivalent*.

Completed HW2 run	Macro F1	Δ vs HW1 TF-IDF	Δ vs best HW1 GloVe
bert-base-uncased	0.5561	+0.1061	+0.1661
distilbert-base-uncased	0.5066	+0.0566	+0.1166
microsoft/deberta-v3-base	0.2672	-0.1828	-0.1228

Table 12: Headline comparison between the completed Homework 2 runs and the main Homework 1 baselines.

7. Subgroup Analysis

The assignment explicitly asks for performance across data subgroups. We therefore analyze the final predictions across short, medium, and long buckets for both question length and answer length.

Question-Length Bucket	Count	Accuracy	Macro F1
Short	107	0.5981	0.5687
Medium	107	0.6168	0.5569
Long	94	0.6809	0.5162

Table 13: BERT performance by question-length bucket.

The BERT model performs worst on long questions and long answers when measured by macro F1. The degradation is especially clear for answer length: short answers reach macro F1 0.6398, while long answers fall to 0.3752. DistilBERT shows a similar answer-length pattern, with macro F1 falling from 0.5132 on short answers to 0.3951 on long answers. This suggests that long responses often contain mixed signals: they may include relevant phrases, hedging, topic shifts, or partial answers, making a single clarity label harder to infer. DeBERTa-v3’s subgroup scores should not be interpreted as meaningful length behavior because the model predicted *Ambivalent* for every test example.

Question-Length Bucket	Count	Accuracy	Macro F1
Short	107	0.5140	0.5010
Medium	107	0.5514	0.4536
Long	94	0.6809	0.5123

Table 14: DistilBERT performance by question-length bucket.

Question-Length Bucket	Count	Accuracy	Macro F1
Short	107	0.5234	0.2290
Medium	107	0.7477	0.2852
Long	94	0.7447	0.2846

Table 15: DeBERTa-v3 performance by question-length bucket. These scores reflect the all-*Ambivalent* prediction collapse.

Answer-Length Bucket	Count	Accuracy	Macro F1
Short	102	0.6569	0.6398
Medium	104	0.5962	0.4666
Long	102	0.6373	0.3752

Table 16: BERT performance by answer-length bucket.

Answer-Length Bucket	Count	Accuracy	Macro F1
Short	102	0.5294	0.5132
Medium	104	0.5769	0.4704
Long	102	0.6275	0.3951

Table 17: DistilBERT performance by answer-length bucket.

Answer-Length Bucket	Count	Accuracy	Macro F1
Short	102	0.5490	0.2363
Medium	104	0.6731	0.2682
Long	102	0.7843	0.2930

Table 18: DeBERTa-v3 performance by answer-length bucket. These scores reflect the all-*Ambivalent* prediction collapse.

8. Error Analysis

The notebook extracts representative mistakes and the most common confusion pairs. This section should keep the final report qualitative but concrete.

8.1. Observed Error Patterns

The BERT run produced 114 errors on 308 test examples. The most common BERT confusion pairs were:

1. *Ambivalent* predicted as *Clear Reply*: 52 cases.
2. *Clear Reply* predicted as *Ambivalent*: 33 cases.
3. *Ambivalent* predicted as *Clear Non-Reply*: 14 cases.
4. *Clear Non-Reply* predicted as *Ambivalent*: 9 cases.

The DistilBERT run produced 130 errors on the same 308 test examples. Its most common confusion pairs were:

1. *Ambivalent* predicted as *Clear Reply*: 58 cases.
2. *Clear Reply* predicted as *Ambivalent*: 32 cases.
3. *Ambivalent* predicted as *Clear Non-Reply*: 24 cases.
4. *Clear Non-Reply* predicted as *Ambivalent*: 8 cases.

The DeBERTa-v3 run produced 102 errors, but this lower error count is misleading: it predicted *Ambivalent* for all 308 test examples. Therefore all 79 *Clear Reply* examples and all 23 *Clear Non-Reply* examples were misclassified.

Thus the main boundary problem for the stable runs is not simply detecting the majority class; it is deciding whether an answer is fully responsive or only partially responsive. *Clear Non-Reply* is weak for both stable completed models, but DistilBERT is more willing to predict this class: it predicts *Clear Non-Reply* 43 times, compared with 32 times for BERT. This increases DistilBERT's *Clear Non-Reply* recall relative to a conservative classifier, but also creates more *Ambivalent* → *Clear Non-Reply* mistakes than BERT.

Gold Label	Pred. Ambivalent	Pred. Clear Non-Reply	Pred. Clear Reply
Ambivalent	140	14	52
Clear Non-Reply	9	13	1
Clear Reply	33	5	41

Table 19: BERT confusion matrix on the official test split. Rows are gold labels and columns are predictions.

Gold Label	Pred. Ambivalent	Pred. Clear Non-Reply	Pred. Clear Reply
Ambivalent	124	24	58
Clear Non-Reply	8	13	2
Clear Reply	32	6	41

Table 20: DistilBERT confusion matrix on the official test split. Rows are gold labels and columns are predictions.

Gold Label	Pred. Ambivalent	Pred. Clear Non-Reply	Pred. Clear Reply
Ambivalent	206	0	0
Clear Non-Reply	23	0	0
Clear Reply	79	0	0

Table 21: DeBERTa-v3 confusion matrix on the official test split. Rows are gold labels and columns are predictions.

8.2. Representative Mistakes

These mistakes are difficult because political interview answers often contain lexical overlap with the question even when they are evasive or only partially responsive. This weakens surface cues and forces the model to judge the relation between the question’s demand and the answer’s actual commitment.

Case	Gold Label	Predicted Label	Short Explanation / Example Snippet
1	Ambivalent	Clear Reply	A question about a political decision received an answer with topical content and institutional framing, but the gold label treated it as only partially responsive.
2	Clear Reply	Ambivalent	A long explanatory answer contained enough hesitation and setup that the model treated it as less direct than the gold label.
3	Clear Non-Reply	Ambivalent	A hypothetical-question answer was rejected by the speaker; the model recognized evasiveness but softened it to the middle class.

Table 22: Representative BERT error types summarized from the notebook examples.

9. Discussion

This Homework 2 design deliberately avoids heavy optimization. That is a feature rather than a weakness. The point of the assignment is to demonstrate understanding of contextual modeling, not to maximize a leaderboard score through large-scale tuning.

The key methodological differences from Homework 1 are:

- raw question–answer pairs instead of hand-normalized text,
- sentence-pair tokenization instead of concatenated sparse features,
- contextual sequence classifiers instead of Logistic Regression,
- one fixed validation split instead of 3-fold cross-validation.

This creates a clean scientific comparison. If Homework 2 improves on Homework 1, the report can reasonably attribute the gain to contextual sentence-pair modeling rather than to a much larger tuning effort.

9.1. Small Non-Improving Changes

The completed sweeps give two compact observations:

- Increasing max length beyond 256 did not help for either completed model. The 384-token settings underperformed the best 256-token setting for both BERT and DistilBERT, suggesting that extra context may introduce noise or that truncation is not the main bottleneck on this split.
- Training for 4 epochs helped when paired with max length 256. BERT rose from 0.6558 at 3 epochs to 0.6657 at 4 epochs, while DistilBERT rose from 0.5742 to 0.6229 under the same max-length setting.
- DeBERTa-v3 did not provide a meaningful tuning signal because every sweep setting produced nan losses and the same validation macro F1 of 0.2476. This points to a numerical stability problem in the run rather than a normal hyperparameter preference.

10. Conclusion

Homework 2 extends Homework 1 in the most direct possible way: it preserves the high-level pipeline but replaces the non-contextual source representation with a contextual transformer encoder and replaces the linear classifier with an end-to-end fine-tuned sequence classifier. This keeps the comparison between the two homeworks clean and easy to explain.

For the stable completed transformer runs, contextual fine-tuning improved over the Homework 1 baselines. BERT reached macro F1 0.5561 and DistilBERT reached macro F1 0.5066, compared with 0.45 for TF-IDF + Logistic Regression, 0.39 for GloVe Wiki, and 0.38 for GloVe Twitter. The best recipe for both stable models was the default-like setting of 4 epochs and maximum length 256. BERT is the strongest valid run. DeBERTa-v3 reached macro F1 0.2672, but this result is not a successful fine-tuning outcome because the losses were nan and all predictions were *Ambivalent*.

The main remaining challenge is the boundary between direct and partially responsive answers. *Clear Non-Reply* is weak for both stable completed models, while the largest number of errors occurs between *Ambivalent* and *Clear Reply*. Long answers are also harder than short answers by macro F1, which suggests that response-clarity classification still requires nuanced reasoning about relevance and evasiveness beyond surface lexical overlap.

References

- [1] AILS-NTUA. Question evasion detection in political interviews. HuggingFace dataset: <https://huggingface.co/datasets/ailsntua/QEvasion>, 2024.
- [2] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics*, pages 4171–4186, 2019.
- [3] Pengcheng He, Xiaodong Liu, Jianfeng Gao, and Weizhu Chen. De-BERTa: Decoding-enhanced BERT with disentangled attention. *arXiv preprint arXiv:2006.03654*, 2020.
- [4] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2017.
- [5] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. Distilbert, a distilled version of BERT: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*, 2019.